

TypeScript: Arrays, Objects & Arrays of Objects

Complete notes, examples, practice questions, and interview-level questions for Node.js + React.

1. Arrays in TypeScript

TypeScript lets you specify what type of values an array can contain.

```
let numbers: number[] = [10, 20, 30];
let names: string[] = ["Niraj", "Rahul", "Amit"];
let isActive: boolean[] = [true, false, true];
```

2. Type Inference

You do not always need to explicitly write the type. TypeScript can infer it from the initial value.

```
let numbers = [10, 20, 30];

numbers.push(40); // OK
numbers.push("hello"); // Error
```

3. Different Array Type Syntaxes

```
let numbers: number[] = [1, 2, 3];
let numbers2: Array<number> = [1, 2, 3];

let names: string[] = ["A", "B"];
let names2: Array<string> = ["A", "B"];
```

Both forms represent the same kind of array. The `number[]` form is generally more concise.

4. Mixed Arrays and Union Types

```
let values: (number | string)[] = [
  10,
  "hello",
  20,
  "world"
];

values.push(50); // OK
values.push("hello"); // OK
values.push(true); // Error
```

5. Common Array Methods

TypeScript keeps normal JavaScript array methods, but it tracks their types.

```
const numbers: number[] = [10, 20, 30, 40];

const doubled = numbers.map((num) => num * 2);
// doubled: number[]

const filtered = numbers.filter((num) => num > 20);
// filtered: number[]

const found = numbers.find((num) => num === 20);
// found: number | undefined

const hasLarge = numbers.some((num) => num > 25);
// hasLarge: boolean

const allPositive = numbers.every((num) => num > 0);
// allPositive: boolean

const total = numbers.reduce((sum, num) => sum + num, 0);
// total: number
```

Important `find()` concept

`find()` can fail to find an element, so its result is normally `T | undefined`. This is a very common TypeScript interview concept.

6. Readonly Arrays

```
const numbers: readonly number[] = [10, 20, 30];

numbers.push(40); // Error

numbers.map(n => n * 2); // OK
numbers.filter(n => n > 10); // OK
```

readonly prevents modification through that reference.

7. Objects in TypeScript

```
const user: {
  name: string;
  age: number;
} = {
  name: "Niraj",
  age: 22
};

user.age = 23; // OK
user.age = "23"; // Error
```

8. Optional Properties

```
const user: {
  name: string;
  age: number;
  address?: string;
} = {
  name: "Niraj",
  age: 22
};
```

The ? means the property is optional and may be omitted.

9. Readonly Object Properties

```
const user: {
  readonly id: number;
  name: string;
} = {
  id: 101,
  name: "Niraj"
};

user.name = "Rahul"; // OK
user.id = 102; // Error
```

10. Objects Containing Functions

```
const calculator: {
  add: (a: number, b: number) => number;
} = {
  add(a, b) {
    return a + b;
  }
};
```

11. Type Aliases

```
type User = {
  name: string;
  age: number;
  email: string;
};

const user: User = {
  name: "Niraj",
  age: 22,
  email: "niraj@example.com"
};
```

A type alias lets you define an object structure once and reuse it.

12. Interfaces

```
interface User {
  name: string;
  age: number;
  email: string;
}

const user: User = {
  name: "Niraj",
  age: 22,
  email: "niraj@example.com"
};
```

For your Node.js + React learning, understand both type and interface. Both can describe object structures.

13. Arrays of Objects

```
type User = {
  name: string;
  age: number;
  email: string;
};

const users: User[] = [
  {
    name: "Niraj",
    age: 22,
    email: "niraj@example.com"
  },
  {
    name: "Rahul",
    age: 23,
    email: "rahul@example.com"
  }
];
```

User means one object. User[] means an array containing User objects.

14. Accessing Objects in Arrays

```
console.log(users[0].name);

users.map(user => user.name);

const adults = users.filter(user => user.age >= 23);

const found = users.find(user => user.name === "Niraj");
// User | undefined
```

15. Nested Objects

```
type Address = {
  city: string;
  pincode: number;
};

type User = {
  name: string;
  age: number;
  address: Address;
};

const user: User = {
  name: "Niraj",
  age: 22,
  address: {
    city: "Mumbai",
    pincode: 400001
  }
};

console.log(user.address.city);
```

16. Arrays Inside Objects

```
type User = {
```

```

    name: string;
    skills: string[];
  };

  const user: User = {
    name: "Niraj",
    skills: ["JavaScript", "TypeScript", "Node.js"]
  };

```

17. Nested Objects + Arrays

```

type Address = {
  city: string;
  country: string;
};

type User = {
  id: number;
  name: string;
  address: Address;
  skills: string[];
};

const users: User[] = [
  {
    id: 1,
    name: "Niraj",
    address: {
      city: "Mumbai",
      country: "India"
    },
    skills: ["Node.js", "React", "TypeScript"]
  }
];

```

18. Real Node.js Example

```

type User = {
  id: string;
  name: string;
  email: string;
  age: number;
};

const users: User[] = [];

function getUserById(id: string): User | undefined {
  return users.find(user => user.id === id);
}

```

This pattern is common in backend services: an array of typed records plus a function that may return one record or undefined.

19. Functions with Arrays of Objects

```

type Product = {
  id: number;
  name: string;
  price: number;
};

function getExpensiveProducts(
  products: Product[]
): Product[] {
  return products.filter(
    product => product.price > 1000
  );
}

```

20. Destructuring Objects

```

const user: User = {
  id: "1",
  name: "Niraj",
  email: "niraj@example.com",
  age: 22
};

const { name, age } = user;

```

```
// name: string
// age: number
users.map(({ name, age }) => {
  console.log(name, age);
});
```

21. Spread Operator

```
const updatedUser = {
  ...user,
  age: 23
};
```

The spread operator is especially important in React because it creates a new object instead of changing the original object.

22. Partial

```
type User = {
  name: string;
  age: number;
  email: string;
};

const update: Partial<User> = {
  age: 23
};
```

Partial makes all properties optional. It is useful for PATCH/update operations where only some fields are supplied.

23. Pick

```
type User = {
  id: string;
  name: string;
  email: string;
  age: number;
};

type UserPreview = Pick<User, "id" | "name">;

const user: UserPreview = {
  id: "1",
  name: "Niraj"
};
```

24. Omit

```
type User = {
  id: string;
  name: string;
  email: string;
  password: string;
};

type PublicUser = Omit<User, "password">;

const user: PublicUser = {
  id: "1",
  name: "Niraj",
  email: "niraj@example.com"
};
```

Omit is useful when creating public API response types that should not expose sensitive fields.

25. Array vs Tuple

```
// Array
let values: (string | number)[] = ["Niraj", 22];

// Tuple
let user: [string, number] = ["Niraj", 22];

user[0] = "Rahul"; // OK
user[1] = 23;      // OK
user[0] = 100;     // Error
```

An array normally represents a collection. A tuple represents a fixed structure where position has a specific type.

Interview-Level Questions & Answers

1. What is the difference between `number[]` and `Array`?

Both represent an array containing numbers. `number[]` is usually more concise; `Array` is the generic form.

2. What does `User[]` mean?

It means an array where every element must follow the `User` type.

3. What does `?` mean in an object type?

It makes that property optional. The property can be present or absent.

4. What does `readonly` mean?

It prevents modification through that typed reference after the value is created.

5. What is the return type of `find()` on `User[]`?

Usually `User | undefined`, because the requested element may not exist.

6. What is the difference between `User` and `User[]`?

`User` represents one `User` object. `User[]` represents an array of `User` objects.

7. What does `(User | null)[]` mean?

It is an array whose elements can be either a `User` object or `null`.

8. What is the difference between `User[]` and `readonly User[]`?

`User[]` can be modified using methods such as `push`. `readonly User[]` cannot be modified through that reference.

9. Why use a type alias or interface?

To define an object structure once and reuse it across variables, functions, API models, components, and services.

10. Why is `find()` different from `filter()`?

`find()` returns one matching element or `undefined`. `filter()` returns an array of all matching elements.

11. What does `Partial` do?

It makes all properties of `User` optional, which is useful for partial updates.

12. What does `Pick` do?

It creates a new type containing only `id` and `name` from `User`.

13. What does `Omit` do?

It creates a type containing all `User` properties except `password`.

14. What is a tuple?

A tuple is a fixed-position structure where each position can have a specific type, such as `[string, number]`.

15. Why are arrays of objects important in Node.js and React?

API responses, database results, lists, tables, and UI collections are commonly represented as arrays of typed objects.

Practice Questions

1. Create an array that accepts only numbers.

2. Create an array containing strings.

3. Create a `Student` type with `id`: `number`, `name`: `string`, and `marks`: `number`.

4. Create an array of `Student` objects.

5. Use `filter()` to find students whose marks are greater than 70.
6. Use `map()` to create an array containing only student names.
7. Use `find()` to find the student whose ID is 2. What is the return type?
8. Create a `Product` type with `id`, `name`, `price`, and `category`, then create an array of products.
9. Find all products whose category is electronics.
10. Calculate the total price of all products using `reduce()`.

Mini Interview Challenge

```
type Employee = {
  id: number;
  name: string;
  department: string;
  salary: number;
};

const employees: Employee[] = [
  {
    id: 1,
    name: "A",
    department: "IT",
    salary: 50000
  },
  {
    id: 2,
    name: "B",
    department: "HR",
    salary: 40000
  },
  {
    id: 3,
    name: "C",
    department: "IT",
    salary: 70000
  }
];
```

1. Get all IT employees using `filter()`.
2. Get employee names using `map()`.
3. Find the employee with ID 2 using `find()`.
4. Calculate total salary using `reduce()`.
5. Get employees earning more than 50,000.
6. Sort employees by salary.

What You Actually Need for Node.js + React

Arrays: type inference, `number[]`, `string[]`, `boolean[]`, union arrays, readonly arrays, `map`, `filter`, `find`, `reduce`, `some`, `every`, and `sort`.

Objects: object type, optional properties, readonly properties, nested objects, methods, type aliases, interfaces, destructuring, and spread.

Utility types: `Partial`, `Pick`, and `Omit`.

Arrays of objects: `User[]`, `Product[]`, `Employee[]`, API response data, and combining `map/filter/find/reduce/sort` with typed objects.

Core Mental Model

```
type User = {
  id: string;
  name: string;
  age: number;
};
```

```
// One User
const user: User = {...};

// Many Users
const users: User[] = [...];

// A User may not exist
const foundUser: User | undefined = ...;

// A User or null
const maybeUser: User | null = ...;

// Partial update
const update: Partial<User> = {...};

// Selected fields
type UserPreview = Pick<User, "id" | "name">;

// Everything except password
type PublicUser = Omit<User, "password">;
```

If you are comfortable with these patterns, you have covered the core arrays/objects knowledge needed before moving deeper into TypeScript for Node.js and React.